



Course Code: CS2009	Course Name: Compiler Construction (CS-4031)
Instructor Name:	Zulfiqar Ali
Date of Submission	29 th March 2026

Instructions:

Max Marks: 100

- Make your Assignments on Full Scale Papers, Typed Assignments will not be acceptable:
- 20% penalty for 1 day late
- 40% penalty for 2 days late
- Submission not allowed afterwards

1. Write a regular expression and then convert it into NFA & DFA for a floating-point number in a language that supports optional sign, decimal points, and scientific notation. The number must have at least one digit before the decimal point if it is present, and the exponent part must have at least one digit. Handle cases where the decimal point or exponent may be omitted
2. Design a deterministic finite automaton (DFA) that recognizes identifiers in a language where identifiers must start with a letter or underscore, followed by any number of letters, digits, or underscores, but cannot end with an underscore. Keywords are reserved (e.g., "if", "else", "while", "for", "switch", "return") and should not be recognized as identifiers. Describe the DFA states and transitions.
3. Consider a language that supports nested comments like /* ... */. Write a lexical analyzer rule and finite automaton to handle nested comments arbitrarily deep. Explain
4. Consider the input string: **int count = 100;**
 - Explain how the two-buffer scheme processes this input step by step.
 - Indicate the positions of both pointers — lexeme-begin and forward — at each step.
 - Identify the lexemes and the corresponding token names.
 - Explain what happens when the forward pointer reaches the end of a buffer.
5. Design a complete lexical analyzer for a subset of C language that handles:
 - Keywords: int, float, if, else, while, return
 - Identifiers
 - Integer and float constants
 - Operators: +, -, *, /, =, ==, !=, <, >, <=, >=
 - Delimiters: (,), {, }, ,, ,
 - Comments (both single and multi-line)

Provide: Regular expressions for each, NFA/DFA diagrams
6. Place CFGs in the Chomsky hierarchy. What is the power of CFGs compared to regular expressions? Give an example of a language that can be expressed with a CFG but **not** with a regular expression.
7. Design a CFG for a simplified JSON object:
 - An object is { } or { members }.
 - Members are comma-separated key : value.

- Key is a string (quoted).
- Value can be string, number, object, array ([elements]), true, false, null.
- Elements are comma-separated values.

(Assume terminal STRING, NUMBER, etc. Do not worry about lexical details.)

8. Write a C Program to recognize a valid variable, which starts with a letter, followed by any number of letters or digits.(Validate Your Name, Student as Variable), also attach the output here.
9. Design a Context-Free Grammar (CFG) for a scientific calculator that includes the following functionalities: basic arithmetic, trigonometric functions (sin, cos, tan), logarithmic functions (log, ln), power and root operations, constants (like pi and e), and parentheses for grouping.
10. Write a C/C++ program to construct a Symbol Table Management (STM) system for a given source code input. The symbol table should categorize and store the following tokens:
Keywords: Store the keyword along with the line number(s) where it appears.
Operators: Store the operator along with the line number(s) where it appears.
Identifiers: Store the identifier name along with its data type (e.g., int, float, char).
Digits/Numeric Constants: Store the numeric value along with the line number(s) where it appears.